

CacheSage-UC: 面向 NutShell Cache 的 UC Agent 辅助自动化验证报告

证据型提交版验证报告

赛题方向: UC Agent / NutShell Cache 自动化验证

GitLink 仓库: gitlink.org.cn/python123/cachesage-uc

GitHub 仓库: github.com/python123-ops/CacheSage-UC

报告身份: python123

提交基线: 仓库当前默认分支 HEAD

报告日期: 2026-06-14

本报告严格基于仓库中可复现的验证证据编写: Python harness、功能覆盖、故障注入、复核记录与 Picker/Toffee 集成边界。报告不声称已经完成真实 RTL/Toffee 覆盖率, 也不声称发现真实 NutShell RTL bug。

目录	1
----	---

目录

1 摘要	2
2 赛题核心任务对照	2
3 验证环境总体架构	3
4 约束随机与场景矩阵	4
5 覆盖率结果	4
6 评审维度证据矩阵	5
7 验证数据完整性	5
8 Scoreboard 设计	6
9 故障注入与 Bug 追踪记录	6
10 故障定位口径	7
11 UCAgent 与人工复核	7
12 Picker/Toffee/NutShell 集成边界	8
13 Linux Smoke 实证记录	9
14 限制与补充条件	9
15 工程复现性	9
16 执行证据摘要	10
17 结论	11
18 附录：仓库与证据文件	11

1 摘要

CacheSage-UC 面向 UCAgent NutShell Cache 赛题，围绕 cache 验证中的数据一致性、事件顺序、replacement、byte mask、stall 与 reset 等高风险路径，构建了一套可复现的验证原型。仓库包含 Generator/CRV、Reference Model、Scoreboard、Coverage Collector、Fault Injection、Review Journal 与 Linux smoke 证据。当前 Python harness 在 seed 11、96 个 transaction 上达到 **23/23**，即 **100.00%** 功能覆盖率，并对 5 类 injected fault 给出确定性检出证据。

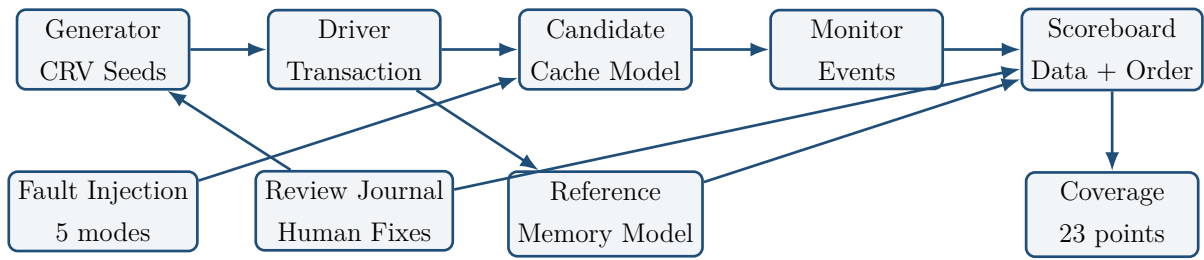
本报告的核心目标是给评审提供可审计材料：哪些能力已经由脚本和测试证明，哪些数据仍需要更完整的 RTL coverage flow 导出。当前 Linux smoke 已跑通 Picker/Toffee/NutShell 的基础链路，但 RTL/Toffee measured coverage 字段仍保留为空值记录；所有 fault artifact 均为 injected fault 检出证据，不代表真实 NutShell RTL 已被确认存在缺陷。

关键词：UCAgent；NutShell Cache；Scoreboard；约束随机；故障注入；覆盖率

2 赛题核心任务对照

任务要求	仓库证据	状态
完整验证组件	Generator、CRV、Scoreboard、Reference Model、Coverage Collector、Fault Injection 均已在 Python harness 中闭合。	已形成
验证报告	本 PDF、reports/initial-verification-report.md、已整理 JSON artifact 与复核记录。	
开发者任务重心	UCAgent 草案不直接落地，关键 invariant、scoreboard 规则 and 报告口径均经人工复核。	有记录
约束细化	same-set pressure、dirty/clean eviction、mask matrix、stall window 与 reset recovery 被拆成明确场景。	已覆盖
架构重构	src/cachesage_uc/adapters/ 记录 Picker/Toffee 风格事务边界。	已建立
故障注入	5 类 injected fault 均有 deterministic artifact。	已检出

3 验证环境总体架构



架构将生成、执行、观测、比对和复核拆开。Generator 负责 directed spine 与 CRV stream; Candidate Cache Model 可切换 fault mode; Reference Memory Model 提供 byte-addressable 期望行为; Monitor 记录 miss、refill、eviction、writeback、stall 等事件; Scoreboard 同时比较 read data、最终 backing memory 与 event signature; Coverage 只统计被 stimulus 与 checker 共同支撑的 coverpoint。

4 约束随机与场景矩阵

场景	验证意图	代表覆盖点
S01	读写冒烟路径，确认 driver、monitor 与 scoreboard 能闭合基本 transaction loop。	read hit, write mask
S02	read miss 与 refill，检查 refill 接收和 replay response 顺序。	read miss, refill alignment
S03	write miss allocate，确认 byte mask 不污染未选中字节。	write miss, mask mix
S04	dirty eviction，在 replacement 前捕获脏 victim 丢写回。	dirty eviction, writeback
S05	clean eviction，确认 clean victim 不产生 phantom writeback。	clean eviction
S06	same-set pressure，触发 replacement policy 状态推进。	replacement rotation
S07	stall/back-pressure，检查 request metadata 稳定性。	stall hold
S08	reset recovery，检查 miss/refill window 中 reset 行为。	reset recovery
S09	边界地址 aliasing，检查 tag/index/offset slicing。	boundary address
S10	长随机回归，补齐 directed tests 未覆盖 interleaving。	long random, multi-set
S11	mask 与 offset 矩阵，暴露 byte-lane 错误。	partial mask, offset
S12	事件级 replacement 审计，检查数据匹配时的 policy drift。	event signature

5 覆盖率结果

指标	当前记录
复现参数	模块 <code>cachesage_uc.cli</code> ；seed=11；count=96；输出 <code>reports/sample-run-seed11.json</code>
transaction 数	96
Python harness 覆盖率	23/23, 100.00%
覆盖点范围	read/write hit、miss/refill、dirty/clean eviction、replacement、mask、stall、reset、boundary address、multi-set traffic
证据边界	RTL/Toffee 覆盖率未实测；不与 Python harness 数据混写

事件计数	数量
dirty_eviction	33
eviction	51
hit	41
masked_write	24
miss	55
read	50
refill	55
reset_window	1
stall_hold	2
write	46
writeback	33

6 评审维度证据矩阵

维度	报告证据	可复核位置
完整性	覆盖 Generator、Scoreboard、Reference Model、Coverage、Fault Injection，并给出 smoke 边界。	src/cachesage_uc/
技术深度	约束随机覆盖 same-set pressure、dirty eviction、byte mask、stall、reset 和 refill alignment。	docs/verification-plan.md
协同效能	review journal 记录草案问题、人工发现、修正方式和 linked evidence。	review_journal.json
工程质量	单元测试、compileall、固定上游 commit、Apache-2.0 license 和双远端同步。	tests/

该矩阵只用于说明证据如何被评审复查，不把项目包装成已经完成真实 RTL 端到端实测。报告中的每个结论都对应到仓库文件、JSON artifact 或可执行命令。

7 验证数据完整性

当前样例 run 的 transaction 规模为 96，事件计数覆盖 miss、refill、write、hit、eviction、dirty eviction、writeback、stall hold 与 reset window。该结果说明 stimulus 不只是普通顺

序读写，而是覆盖了 cache data path 与 control path 的组合。特别是 dirty eviction 与 writeback 同时出现，使 scoreboard 能检查 victim 数据是否在 replacement 前写回；stall hold 与 reset window 的出现，则为 handshake stability 与 reset recovery 提供了可执行证据。

JSON artifact 采用机器可读格式保存，便于评审重新运行命令后比对字段：passed、coverage、covered_points、event_counts 和 failures。报告生成脚本直接读取这些字段生成表格，减少人工复制导致的数字漂移。

8 Scoreboard 设计

Scoreboard 不只比较最终读数据，而是把 cache 正确性拆成三类约束。第一类是数据约束：read response 与 reference memory model 对齐，masked write 必须保留未选中 byte lane。第二类是状态约束：最终 backing memory 必须包含 dirty victim 的写回结果。第三类是事件顺序约束：writeback、refill、eviction 与 stall-tagged metadata 的 event signature 必须匹配预期。

这种设计针对 cache 验证中的常见盲点：一个短 readback 可能掩盖 dirty writeback 顺序错误；普通随机流可能无法稳定触发 replacement 状态漂移；full-word store 可能隐藏 byte mask 错误。因此，报告中的覆盖率不是单纯 stimulus 命中率，而是 stimulus 与 checker 同时闭合后的功能覆盖记录。

9 故障注入与 Bug 追踪记录

fault mode	检出结果	failures	首个失败摘要
drop_dirty_writeback	预期失败	6	memory mismatch at 0x0: expected 0x01020304, observed 0x00000000
ignore_write_mask	预期失败	4	memory mismatch at 0x4: expected 0x0000CCDD, observed 0xAABBCCDD
refill_shift	预期失败	13	memory mismatch at 0xC: expected 0x00000000, observed 0x11223344
stuck_replacement	预期失败	6	memory mismatch at 0x20: expected 0x55667788, observed 0x00000000
unstable_under_stall	预期失败	2	data mismatch at 0x0: expected 3405691582, observed 3405691457

上述 5 类 fault 均为 injected fault，用于验证环境本身的检出能力。其中 drop_dirty_writeback 保护 dirty victim 写回，ignore_write_mask 保护 byte lane 语义，stuck_replacement 保护 replacement 状态推进，refill_shift 保护 refill beat 对齐，unstable_under_stall 保护 handshake 稳定性。报告不声称已发现真实 NutShell RTL bug。

10 故障定位口径

每个 injected fault 的价值不在于制造失败，而在于验证 scoreboard 是否能给出可解释的失败信号。报告采用三层定位口径：第一层检查 read response 或 final memory 是否 mismatch；第二层检查 monitor event 是否出现缺失、错序或地址漂移；第三层把失败归因到 stimulus、scoreboard 或 candidate model。只有在三层信息一致时，报告才把该 fault 记为有效检出。

这种口径能避免两个常见问题：一是把刺激生成错误误判为 DUT 错误；二是只看最终数据而忽略 writeback/refill 的事件顺序。CacheSage-UC 当前的 fault artifact 已经覆盖数据、控制、replacement 与 stall 四类风险，因此可作为后续 RTL smoke 的诊断模板。

11 UCAGENT 与人工复核

ID	复核发现	修正方式	覆盖变化
RV-001	替换策略和 dirty victim 没有被单独拆开，容易把最关键的数据一致性问题藏在随机流里。	补入 directed dirty eviction、clean eviction、same-set pressure，并让 scoreboard 检查 writeback 事件。	新增替换与脏写回覆盖
RV-002	均匀分布很难快速打到同 set 冲突，也不利于复现 replacement failure。	将 seed 前缀做成 directed coverage spine，随机段再混入 word offset、mask 和多 set 访问。	新增 same-set、mask、offset 覆盖
RV-003	这会让作品看起来像普通脚本，缺少 cache 控制路径和时序保持类风险。	补充 stuck_replacement、refill_shift、unstable_under_stall，并为每个 fault 固定 deterministic seed。	fault mode 从 2 类扩展到 5 类

ID	复核发现	修正方式	覆盖变化
RV-004	这个口径会在评审追问 Toffee 证据时露怯。	报告拆成 planned coverage、Python harness measured coverage、RTL/Toffee measured coverage 三栏，真实...	报告证据分栏
RV-005	没有读取上游工程结构就定义接口，后续很可能与 Picker/Toffee 真实路径对不上。	锁定 XS-MLVP/Example-NutShellCache commit，新增 fetch 脚本和 layout inspector，只在本地拉取第三方源码。	集成边界可复现

复核记录体现了 UCAgent 的实际作用边界：草案生成可以提高启动速度，但 cache invariant、scoreboard 规则、覆盖率口径和上游接口假设必须由人工复核。当前仓库保留 `review_journal.jsonl`，记录 prompt、draft summary、review finding、correction 与 linked evidence，使报告中的设计修正能够追溯到测试或文档。

12 Picker/Toffee/NutShell 集成边界

项目	当前记录
上游工程	XS-MLVP/Example-NutShellCache, commit 固定于 cdc9ef7d4dfc3d8fbd969869f6696afe27cfed2a
本机 smoke 状态	rtl_smoke_complete
缺失依赖	无
依赖说明	Linux 环境依赖齐全；上游 make gen_dut 与 make test smoke 已通过。
RTL/Toffee 覆盖率	RTL/Toffee 覆盖率未实测，字段保留为空值记录

该边界是有意保守的：Linux 环境已完成上游 `make gen\_dut` 与 `make test smoke`；RTL/Toffee measured coverage 尚未由上游测试导出。报告把环境 smoke 通过、Python harness 覆盖率和 RTL measured coverage 分栏记录，避免把尚未导出的覆盖率写成实测结论。

13 Linux Smoke 实证记录

本次 Linux smoke 运行在 Ubuntu 24.04 WSL2 环境中，系统依赖包含 make、CMake、GCC/G++、Verilator、SWIG 与 Python venv。Picker 安装后 `picker--check` 显示 C++ 与 Python 支持可用；Python venv 中固定 `pytoffee==0.3.0` 与 `toffee-test==0.3.0`，避免 PyPI 最新包之间的接口漂移。

上游 Example-NutShellCache 的 `makegen\_dut` 返回 0，完成 Picker 生成 Python DUT 与 Verilator build；`maketest` 返回 0，pytest 记录为 `test/test\_smoke.py::test\_smokePASSED`。该证据说明基础环境构建已经从准备态变为可执行 smoke，但还没有把上游 coverage 数据导出为可量化 RTL measured coverage。

14 限制与补充条件

本报告的主要限制不再是基础环境缺失；当前 Linux smoke 已经证明 Picker 生成 DUT、Toffee 测试入口和上游 pytest smoke 能够跑通。仍需补充的是由上游测试或后续 coverage flow 导出的 RTL/Toffee measured coverage、waveform 片段和更长随机回归。

补充真实 RTL 证据时，应沿用当前场景矩阵和 seed 策略：先运行上游 `makegen\_dut` 与 `maketest`，再把 Picker-generated DUT 接入同一 driver/monitor/scoreboard 路径。新增数据应至少包括 RTL measured coverage、失败 transaction trace、关键 waveform 截图或路径，以及人工复核结论。

15 工程复现性

复现项	命令或材料
安装	<code>python -m pip install -e .</code>
测试	<code>python-munittestdiscover-stests-v</code>
编译检查	<code>python-mcompileall-qsrctestsscripts</code>
计划导出	<code>python-mcachesage_uc.cliplan</code>
覆盖率样例	<code>python-mcachesage_uc.clirun--seed11--count96--outputreports/sample-run-seed11.json</code>
报告构建	<code>pythonscripts/build_verification_pdf.py</code>
许可证	Apache License 2.0

16 执行证据摘要

1. 基础测试: `python-munittestdiscover-stests-v`, 覆盖 CLI、证据模型、公开材料约束、上游适配和 cache verification core。
2. 编译检查: `python-mcompileall-qsrc-testscripts`, 用于确认源码、测试和脚本均可被 Python 编译器解析。
3. 覆盖率样例: `reports/sample-run-seed11.json` 记录 seed、transaction count、covered points 与 event counts。
4. smoke 记录: `reports/nutshell-smoke.json` 记录上游目录、`makegen\_dut`、`maketest` 与 `pytest smoke` 均已通过。
5. 报告构建: `scripts/build_verification_pdf.py` 从 JSON/JSONL 证据生成 Markdown、LaTeX 与 PDF, 减少手工维护偏差。

这些证据共同构成提交包的可复核路径: 评审可以先阅读 PDF, 再沿着报告中的文件路径和命令回到仓库复现核心数据。若后续加入真实 RTL/Tofee 运行结果, 仍应沿用同样的证据链格式。

17 结论

CacheSage-UC 当前已经形成面向 NutShell Cache 的可复现验证仓库和正式验证报告。仓库中可运行证据覆盖了 Generator/CRV、Scoreboard、Reference Model、Coverage 与 Fault Injection; seed 11 的样例回归达到 **23/23** 功能覆盖点; 5 类 injected fault 均被确定性检出; 人工复核记录说明了草案问题如何被修正为可审计的工程验证逻辑。

当前 Linux smoke 已进一步证明基础 Picker/Toffee/NutShell 环境链路可运行: 上游 makegen_dut 与 maketest 均返回成功, pytest smoke 为 1 passed。后续工作应集中在导出 RTL measured coverage、保存关键 waveform 片段, 并把更长随机回归接入同一证据链。

18 附录: 仓库与证据文件

- README.md: 项目入口、复现命令和证据边界。
- docs/verification-plan.md: 12 个验证场景与覆盖策略。
- docs/scoreboard-design.md: Scoreboard invariant 与 Toffee 映射。
- docs/fault-injection.md: 5 类 injected fault 的检测目标。
- reports/sample-run-seed11.json: 23/23 覆盖率样例。
- reports/fault-*.json: 故障注入 artifact。
- review_journal.jsonl: prompt、草案、复核发现、修正与证据链接。
- upstream.lock.json: Example-NutShellCache 固定来源信息。